

ME-Box: A reliable method to detect malicious encrypted traffic

Bingfeng Xu^a, Gaofeng He^{b,c,*}, Haiting Zhu^b

^a College of Information Science and Technology, Nanjing Forestry University, Nanjing, 210037, China

^b College of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing, 210003, China

^c Key Laboratory of Computer Network and Information Integration (Southeast University), Ministry of Education, China

ARTICLE INFO

Keywords:

Encrypted traffic
Malicious detection
DPI
Machine learning
Evidence

ABSTRACT

Currently, encryption (such as the Transport Layer Security protocol) is used by increasingly more network applications to protect their security and privacy, while it also benefits network attackers who can encrypt their traffic to evade detection. The detection of malicious encrypted traffic is becoming a critical task for cyber security. To accomplish this task, researchers have proposed several enlightening methods, including decryption followed by deep packet inspection (DPI), direct DPI on ciphertext and identification by machine learning algorithms. However, due to privacy violations or performance limitations, the state-of-the-art is far from satisfactory.

In this paper, we propose a novel framework and system called ME-Box (Machine learning and Evidence verification) for reliable detection of malicious encrypted traffic. ME-Box has middleboxes deployed in the network and agents installed on the sending hosts. Middleboxes first evaluate the trust degrees of encrypted flows by *machine learning* methods. If some flows are classified as suspicious, then middleboxes provide *evidence* of the evaluation results and request the corresponding session-keys from the agents. The agents verify the evidence, and if it is convincing, respond with the correct session-keys. With the session-keys, middleboxes finally decrypt the suspected encrypted flows and perform conventional DPI using intrusion signatures. We implement a prototype system of ME-Box and test it with real malware traffic. The experimental results show that ME-Box requires no modification of current cryptographic protocols and keeps end-users' privacy well, and its performance is practically deployable.

1. Introduction

Currently, increasingly more network applications are adopting encryption protocols such as TLS (Transport Layer Security) to protect their security and privacy. For instance, over 72% of Internet traffic was encrypted in 2018, as reported by Fortinet Networks [1]. Although application users' security and privacy can be better protected, the growth of encryption presents severe challenges to network monitoring and network attack detection. An attacker can use encryption to obscure their presence and evade detection because conventional DPI technology only works on plaintext data. Malware *Linux.Dofloo* is such an example. Its communications to the C&C server were encrypted by the AES algorithm; thus, real-time network monitoring was totally ineffective [2]. Actually, it takes companies, on average, between 100 and 200 days to detect an attack due to traffic encryption [3].

To detect malicious encrypted traffic efficiently, researchers have proposed several enlightening methods, including decryption followed by DPI [4,5], direct DPI on ciphertext [6–8] and machine learning approaches [9–11]. These methods are usually carried out by middle-

boxes operated by network managers or Internet Service Providers. Decryption followed by DPI methods are straightforward. They first decrypt all encrypted flows and then perform malware detection (an advanced DPI function) on the plaintext data. Their detection results are accurate. However, the privacy of end-users is seriously threatened since the middleboxes can observe all communication contents.

Direct DPI on ciphertext can protect end-users' privacy well. The basic idea of these methods is to *tokenize*, *encrypt* and *match* keywords and traffic. When an end-user sends a packet, she/he additionally needs to transform the strings of the packet payload into a series of tokens. The DPI rule generator (e.g., a network security company) also needs to tokenize its rules into keywords. These tokens and keywords are further encrypted and sent to the middlebox for matching. If an encrypted token matches an encrypted keyword, alarms of malicious encrypted traffic are raised. The matching is performed on the encrypted data; thus, the privacy protection of end-users can be guaranteed. However, these methods have serious performance issues. For example, the sender has to encrypt and send the traffic contents

* Corresponding author at: College of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing, 210003, China.

E-mail addresses: bingfengxu@njfu.edu.cn (B. Xu), hegaofeng@njupt.edu.cn (G. He), htzhu@njupt.edu.cn (H. Zhu).

twice, i.e., encrypt the packets with a standard cryptographic protocol such as TLS and send them to the receiver, and then encrypt the tokens that are later sent to the middlebox. This generates heavy encryption and communication overheads [12].

In addition to DPI technologies, machine learning approaches can also be utilized to detect malicious encrypted traffic. By collecting enough normal and malicious encrypted traffic, one can train classification models with typical machine learning algorithms. With the trained models, malicious encrypted traffic can be efficiently classified and detected. Anderson et al. [11] selected the flow metadata, the sequence of packet lengths and interarrival times, the distribution of bytes, TLS information and self-signed certificates as the classification features, and obtained over 96% detection accuracies for the 18 tested malware families. Using deep learning, Chen et al. [13] achieved a 99.63% detection accuracy for malicious flows. However, the rates of false positives of machine learning approaches are also high and restrict their practical applications [14].

In this paper, we propose a novel framework and system called ME-Box (Machine learning and Evidence verification) for *reliable detection* of malicious encrypted traffic. Reliable detection means that the method should be *accurate* (with a high detection rate and almost no false positives), *privacy-preserving*, *low-overhead* and *compatible* with existing security protocols. To this end, in the ME-Box system, the middlebox first uses machine learning algorithms to determine the trust degree of an encrypted flow. If the flow is classified as suspicious, then it sends the *evidence* to the end-user¹ and requests the session-key of the corresponding flow. The end-user receives and verifies the evidence, and if the verification is passed, she transmits the session-key to the middlebox. With the key, the middlebox can finally decrypt the suspicious flow and check the contents with the existing intrusion detection systems such as Snort and Suricata.

Especially in our design, the middlebox must provide evidence for its classification results. This is because we argue that ***the used machine learning methods could be malicious and threaten end-users' privacy***. For example, a middlebox can deliberately classify some specific normal flows as malicious, which can be realized by adding some extra rules for the classification results easily or by inserting a backdoor into the classification models stealthily [15], to obtain session-keys and decrypt the contents. In ME-Box, we introduce a verification server to prevent such dishonest middleboxes and several other attacks and protect end-users' privacy well. Notably, as further described in Section 3.1 and discussed in Section 7, the introduced verification server can additionally provide solutions for some practical machine learning issues, such as imbalanced network traffic and unfair comparison in the network security area [16].

The highlights of the proposed method are as follows. We use machine learning approaches to ensure the detection rate and decrypt suspicious flows to reduce false alarms. This fits the *accuracy* goal. Meanwhile, end-users only need to reveal the session-keys of possible malicious flows confirmed by themselves. Therefore, this minimizes the risk to end-users' privacy, and it is *privacy-preserving*. Evidence sending and verification will generate overhead. We design a data structure similar to that of the X.509 certificate to contain the evidence, and its verification only takes a few steps. Our experiments show that the introduced overhead is low and has little impact on the endpoints. The whole process runs as an independent system, so it is *compatible* with existing security protocols. Our main contributions are as follows:

- (i) Providing a novel framework for detecting malicious encrypted traffic. In the proposed framework, we first use machine learning algorithms to classify encrypted flows. If they are labeled as suspicious, we then decrypt them and further check the contents

to make the detection more accurate. Additionally, the end-users' privacy can be well protected since only possible malicious flows are decrypted.

- (ii) Raising the issue of malicious machine learning methods and outlining an efficient method to verify their classification results. In ME-Box, the classification results are checked by several different machine learning models deployed at a verification server to ensure that they are not detrimental to end-users' privacy. Notably, the verification server is equipped with a continuously growing dataset of labeled malicious encrypted flows and can be an open platform for comparing different machine learning models.
- (iii) Implementing a prototype system of ME-Box and carrying out extensive experiments to validate its performance.

The rest of this work is structured as follows: In Section 2, we introduce the relevant related work. Section 3 illustrates the architecture of ME-Box. The methodology is presented in Section 4 and analyzed in Section 5. An experimental evaluation of ME-Box from the performance standpoint is described in Section 6. We give the discussion of our method in Section 7. Finally, in Section 8, we conclude the paper with a discussion of potential future work.

2. Related work

In this section, we introduce recent studies that address security requirements in the context of encrypted network traffic.

Decryption followed by DPI. The most straightforward way to check the security of encrypted traffic is to decrypt it. After decryption, one can obtain the plaintext data and conveniently carry out DPI with various kinds of detection signatures. Hence, researchers have developed several solutions for middlebox decryption on specific protocols. TLS transparent proxy [17,18] is such an example. The transparent proxy usually acts as a man-in-the-middle (MITM) proxy to split the encrypted connection into two parts: client-to-proxy and proxy-to-server. The proxy first imports its root certificate into the client's trusted certificate authority stores. When a TLS connection is initiated by a client application (e.g., a browser or email client) to a remote server, the proxy forges as the server to complete the TLS protocol. Meanwhile, it starts a second TLS connection to the remote server. Then, the proxy can inspect messages between the two connections to detect malicious contents. Despite the easy implementation and deployment, the transparent proxy breaks the end-to-end encryption and has been found to damage users' security and privacy seriously [19].

mcTLS [4] is the first nontransparent proxy for the TLS protocol. It provides a certificate-based authentication mechanism by which the client and the server can authenticate the middlebox (i.e., the proxy). It also designs a new handshake protocol to support the sharing of session-keys and context keys. However, mcTLS needs to change the existing client, server, and middlebox software, which makes it impractical for now. Liu et al. [5] proposed PlainBox to enable session-key sharing among the endpoints and the middlebox. Unlike mcTLS, PlainBox does not need to modify the standard cryptographic protocols. It uses out-of-band message exchanges for both middlebox authentication and key sharing. Therefore, it is more practical in practice. However, end-users' privacy cannot be well protected as the middlebox can decrypt every encrypted flow.

Direct DPI on ciphertext. To protect end-users' privacy, researchers have proposed several methods to perform DPI on ciphertext directly. BlindBox [6] was the first system to achieve this functionality. In BlindBox, before sending a message, the message is tokenized by a sliding window algorithm. Thus, for every offset in the byte stream, the sender creates a token of a fixed length. The token is encrypted by the AES algorithm with a key k and a salt s . The detection rule r is tokenized and encrypted by the middlebox. Particularly, the encryption of r is accomplished by Yao garbled circuits [20] because

¹ In ME-Box, only the sending host will be involved in the process. Therefore, the terms *end-user*, *endpoint* and *sender* will be used alternatively in the following.

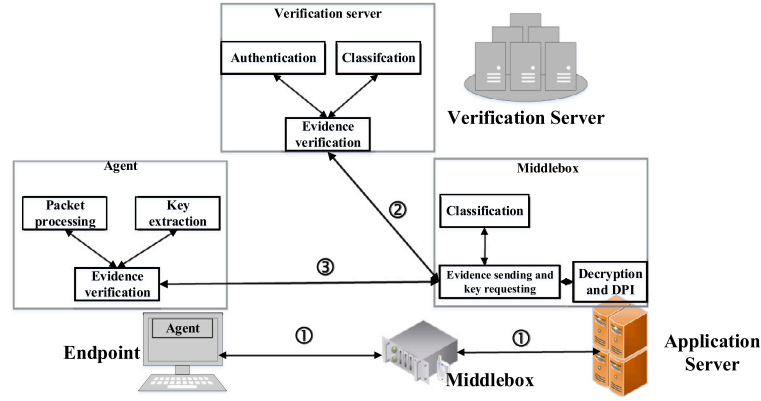


Fig. 1. Architecture of ME-Box.

the middlebox is not allowed to know the key k . DPI is performed by matching the encrypted rules and encrypted tokens. Similar work is also presented in [21]. Canard et al. [7] proposed BlindIDS to perform DPI directly on encrypted traffic. BlindIDS uses the Decryptable Searchable Encryption (DSE) cryptographic tool [22] to encrypt the traffic content and the detection rules. Compared to BlindBox, BlindIDS encrypts the traffic content only once, hence the performance is improved.

Recently, Li et al. [8] used a homomorphic encryption method to achieve privacy-enhanced DPI. To be specific, the Boneh–Goh–Nissim cryptosystem [23] is adopted to encrypt the tokenized messages and the keywords of rules. An encrypted token t and an encrypted keyword w are sent to a server A to calculate the discriminator $d = t \cdot w$. The value of d is further sent to another server B for checking. If $d = d^p = 1$, a match is found, where p is the shared key between B and the rule generator. In one study [24], the authors proposed searchable encryption with shiftable trapdoors. It allows for pattern matching with universal tokens; in other words, keywords of arbitrary lengths can be matched to arbitrary ciphertexts. However, the pattern-matching process is slow.

Machine learning methods. There have been extensive works on detecting malicious encrypted traffic by machine learning methods. Literatures [25,26] give surveys of recent works. Hellemons et al. [27] proposed a flow-based SSH intrusion detection system SSHCure. It is based on a three-phase state machine that monitors the packets-per-flow and the minimum number of flow records. Barati et al. [28] used flow-based features instead of packet features to solve the encrypted traffic IDS problem. They constructed a hybrid model of the Genetic algorithm and Bayesian network classifier. He et al. [29] constructed cloud users' network behavior profiles to detect encrypted data exfiltration. Similarly, Koch et al. [30] also used behavior-based detection architecture to detect intrusions as well as insider activities such as data exfiltration in encrypted environments. Aceto et al. [31–33] adopted deep learning methods to classify mobile encrypted traffic and compared different state-of-the-art deep learning techniques in a systematic framework. Literature [34] selected detection features from machine learning models themselves.

Since TLS has become the de facto standard for secured Internet communications, it is also widely used by attackers to encrypt their data transformations [35]. Therefore, it is meaningful to go deep into malware's TLS traffic and design efficient detection methods. Anderson et al. [9–11,36] have done extensive work on this. By selecting the flow metadata, the sequence of packet lengths and interarrival times, the distribution of bytes, TLS information and self-signed certificates as the classification features and using logistic regression as the classification algorithm, they achieved a detection accuracy over 96% [11]. Despite the high detection rate, challenges still exist because of the inaccurate ground truth and highly nonstationary data distribution [10]. The reduction of false alarms when adopting machine learning methods in network security protection is urgently needed [14,37].

3. Overview

This section presents an overview of the proposed ME-Box, including its main architecture, design goals, and threat model. The detailed designs are elaborated in Section 4.

3.1. Architecture

Fig. 1 shows the architecture of ME-Box. Generally, there are four main individuals and three main connections in the system. The four individuals are the endpoint, middlebox, application server, and verification server. The endpoint communicates with the application server as usual (connection ① in Fig. 1), and the messages can be encrypted. The middlebox locates in the network path, and it can observe all communications between the endpoint and the application server. It analyzes the (encrypted) traffic to determine malicious attacks. Note that in a real network, there can be several middleboxes deployed by different organizations for security checking, and their functions and procedures are the same.

The verification server and the agent are responsible for evidence generation and verification. The agent is installed on the endpoint. It checks network connections locally, receives evidence from the middlebox, and returns the requested session-keys (connection ③). The added load of the agent is light because the verification server is to do the most arduous evidence verification work. The verification server validates middlebox's detecting results and generates the evidence by running the corresponding classification models. It also provides interfaces for middlebox's registration and suspicious flows uploading (connection ②). There can be several verification servers to balance load. In the following sections, we will use only one for illustration.

Illustration example. Suppose that the endpoint has been infected by a malware program, and the malware program communicates with its application server (a malicious server in this example) through an encrypted network connection f . The middlebox observes f and uses machine learning algorithms to classify it. The classification result is positive (In the following, we will always assume that malicious traffic is the positive sample, and normal traffic is the negative sample); thus, the middlebox constructs a piece of incomplete evidence EV and sends it with the flow f to the verification server. The incomplete evidence EV contains some essential information, such as f 's hash value, the destination IP address and port of f , and the classification model label. Besides, EV currently has some blank fields, such as the verification results and signatures that are supposed to be filled out by the verification server.

The verification server reads the classification model label from EV and adopts the corresponding model to classify f . Note that the middlebox has uploaded its *encrypted* classification models to the verification server in the initialization step (the details are in Section 4.1).

Consequently, the verification server can invoke the corresponding model. If the result is positive, one can believe that the middlebox did use the classification model and it did not generate the classification result arbitrarily. However, at this point, anyone should not fully trust the middlebox. This is because the adopted classification model could be very simple or even malicious—for example, it judges every encrypted flow as suspicious. Therefore, the verification server further uses several different models to classify f and saves all the classification results into EV .

At last, the verification server signs EV by its private key to ensure the authenticity and integrity of evidence and returns the complete EV to the middlebox. The middlebox forwards EV to the endpoint and asks for the corresponding session-key. If the major classification results are positive, the endpoint chooses to trust the middlebox and responds with the correct session-key. With the session-key, the middlebox decrypts f and inspects the plaintext contents. The middlebox also feeds back the final DPI detection result to the verification server to label f accurately.

As described in the above text, we mainly use the verification server to ensure that the middleboxes are honest. Especially, the verification server can be an open platform for comparing different machine learning models for network security. In ME-Box's design, the middlebox needs to send the suspicious encrypted flows and the final detection results to the verification server. This implies that the server will have a continuously growing dataset of labeled encrypted flows (malicious or not). This dataset can be used to solve the problem of unbalanced data in the network [38] and compare different detection models fairly. Hence, it may promote the adoption of machine learning algorithms in network security protection. We also discuss a non-central based solution for classification results validation in Section 7.

3.2. Design goals

We recognize the key design goals of ME-Box as below:

- Accurate. It should have high detection rates and almost no false positives. Only in this way can it be deployed in real situations.
- Low-overhead. The system should not add significant overheads to the endpoint so that it can be supported by limited resource equipment such as mobile devices or sensors.
- Good compatibility. The system should be compatible with existing security protocols. Specifically, current security protocols do not need to be modified for adaptation to ME-Box.
- Privacy-preserving. The end-users' privacy must be well protected. Hence, only the suspicious flows confirmed by endpoints should be further processed. Moreover, the session-keys are most critical, and they can only be shared with the appropriate middlebox. Any other third-party, including the verification server, should not obtain these session-keys. Besides the protection of end-users' privacy, the privacy protection of middleboxes is also important. Specifically, the detection models should be encrypted so that the verification server cannot learn any implementation details from them.

3.3. Threat model

In this paper, we consider a new threat model for the usage of machine learning in the network security area: the machine learning algorithms could be malicious and attempting to access end-users' privacy information. For instance, a malicious algorithm judges some specific encrypted flows (e.g., connections to *Facebook*) as suspicious so that network administrators can acquire users' sensitive social behaviors. To overcome this threat, we introduce a new concept—evidence of results and use third-party verification servers to validate the functions of machine learning algorithms.

Consequently, the verification servers and the endpoints are assumed to execute the ME-Box protocols strictly, while the middleboxes

Table 1

Notation used in this paper.

Notation	Meaning
EV	evidence
E	the endpoint
A	the agent installed on the endpoint
M	the middlebox
V	the verification server
$IdentM$	the identity of middlebox M
$XCertV$	the X.509 certificate of V
$XCertM$	the X.509 certificate of M
$ProofCert(f)$	the certificate contains evidence of suspicious flow f
$PU(M)$	the public key of M
$PR(M)$	the private key of M
$PU(V)$	the public key of V
$PR(V)$	the private key of V
$Enc(.,.)$	symmetric encryption function such as AES
$Sig(.,.)$	digital signature function such as RSA
$Hash(.)$	hash function such as SHA512

can be dishonest. They may be interested in the traffic content for commercial or personal benefits. Therefore, the middleboxes may send fake evidence to swindle session-keys from endpoints. We also assume that all session-keys are safely used by the endpoints and middleboxes, which means that all keys are stored encrypted and to be destroyed in time. One can use Intel SGX [39] to satisfy this assumption.

An attacker can monitor endpoints' network communications passively. Additionally, the attacker can masquerade as a middlebox to communicate with the endpoint. He can also occupy middleboxes and modify their functional components, for example, by replacing the original classifiers with the self-designed ones. Active attacks against the endpoints and verification servers are also allowed in ME-Box. For example, an attacker can occupy the endpoints or verification servers through software vulnerabilities or social engineering. After that, he can try to steal data or cheat other system participants. However, we assume that all of the private keys are kept securely and cannot be accessed by attackers.

4. Methodology

ME-Box uses machine learning approaches to detect suspicious encrypted traffic, verification servers to validate evidence, and DPI on plaintext to confirm detections. In this section, we elaborate on its design details in the order of these processing steps. To be specific, the critical technical details include how to initialize the system, how to send evidence to the endpoint and validate it, and how to extract applications' session-keys efficiently. Table 1 defines the notation we use in this paper.

4.1. Initialization

There is no special initialization requirement for the endpoint and the agent. The agent should be installed with the verification server's X.509 certificate $XCertV$. It also needs to establish a long-lived TLS connection with the verification server upon start-up. The initialization steps for the middlebox are a little more complicated. When the middlebox starts for the first time, it must be registered at the verification server. For registration, the middlebox's owner first generates a public key pair: a public key $PU(M)$ and a corresponding private key $PR(M)$. The private key $PR(M)$ is kept secretly, and the public key $PU(M)$ is open. Then, the owner selects a name as the identity of the middlebox. Finally, the identify $IdentM$, the public key $PU(M)$, and the name of the digital signature algorithm are sent to the verification server to complete the registration process.

After registration, the middlebox continues to upload its classification models to the verification server. The middlebox may have several models for different service levels. For each model, it generates a

locally unique number as the label. Classification models may also have related feature extraction modules. Therefore, the middlebox sends the triple $\langle \text{label}, \text{classification_model}, \text{feature_extraction_module} \rangle$ and a *signature* value to the verification server to finish the initialization process. This can be illustrated as formula (1). In formula (1), the number 1 is the label, and sig_1 is the signature value of the triple. This value is calculated by encrypting the hash value of $\langle \text{label}, \text{classification_model}, \text{feature_extraction_module} \rangle$ with the private key $PR(M)$, as shown in Eq. (2).

$$M \rightarrow V : \text{Enc} \left(\left\langle 1, \text{classification_model}_1, \text{feature_extracion_module}_1, \text{sig}_1 \right\rangle, \dots \right) \quad (1)$$

$$\text{sig}_1 = \text{Sig} \left(\text{Hash}(1, \text{classification_model}_1, \text{feature_extracion_module}_1), PR(M) \right) \quad (2)$$

Once the verification server receives the uploaded triples, it verifies the signatures according to M 's public key $PU(M)$. If the signatures are accepted as valid, the classification models and the feature extraction modules are deployed for later evidence verification. Note that the initialization of the middlebox is performed at the beginning. Thereafter, the middlebox only needs to build a secure TLS connection with the verification server upon start-up.

Remark. In practice, the classification models would be trade secrets, and the middlebox could be unwilling to upload them to the verification server. To alleviate this issue, the middlebox's owner can first repack the classification models and the feature extraction modules as a binary program, which is then to be uploaded and deployed. Furthermore, the owner can use indistinguishability obfuscation and functional encryption [40] to prevent the analysis of the program. Since the models are encrypted, the verification server cannot get any implementation information from them. We will describe a practical encryption method in the following subsection.

4.2. Machine learning detection and evidence construction

When the middlebox completes the initialization process and starts to work, it uses machine learning approaches to estimate the trust degree of encountered encrypted flows. In our work, the trust degree has only two possible options: *normal* or *suspicious*. If an encrypted flow is classified as positive (note that we consider the malicious traffic as the positive sample and the normal traffic as the negative sample in this work), it is labeled as suspicious. Otherwise, the flow is considered normal and trusted. To optimize the estimation procedure, one can use *classification confidence* [41] to produce more refined levels. For example, if the classification confidence exceeds a set threshold, the flow will be labeled as *malicious* and does not need to be further decrypted. However, the method of choosing an accurate threshold is complicated and experience-related in practice, so we consider only two degrees for general purpose.

The selected machine learning approaches can be arbitrary, except that the classification models can be encrypted. For instance, one can choose SVM (Support Vector Machine) as the classifier and use the Boneh–Goh–Nissim cryptosystem to encrypt the classification model [8]. Also, secure multi-party computation can be explored to encrypt the deep learning models [42]. However, these methods are model-specific, and they all need to encrypt the models and the classification features simultaneously. In ME-Box, the classification features do not need to be encrypted because they are extracted and utilized by the same subject (i.e., the middlebox). Therefore, in this work, we use the popular industry approach [43] to encrypt classification models. We first encrypt the trained models by some symmetric encryption algorithms such as AES and then decrypt the models when they are invoked. In this way, ME-Box can support all kinds of machine learning algorithms, and the details of classification models can be protected well.

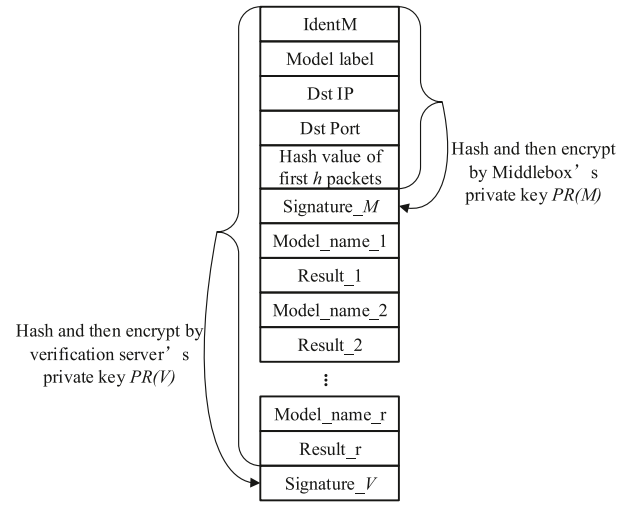


Fig. 2. The structure of ProofCert used in ME-Box.

If an encrypted flow f is judged as suspicious, the middlebox M sends f with a special data structure *ProofCert* to the verification server for constructing a piece of integrated evidence. Actually, the evidence is organized into a data structure that is similar to that of the X.509 certificate and is termed as *ProofCert* in ME-Box. The structure of *ProofCert* is depicted in Fig. 2. The first 6 fields are filled out by the middlebox before sent to the verification server. Among them, *IdentM* is the identity of the middlebox M , and *Model label* is the label of the classification model used by M to estimate f . *Dst IP*, *Dst Port* and *Hash value of first h packets* represent the destination IP address, destination port and hash value of contents of the first h packets of f . These values are signed by M to ensure their authenticity and integrity.

Once the verification server receives f and the corresponding *ProofCert*, it begins the evidence checking and supplementing process. First, it checks *IdentM* and the value of *signature_M* by the middlebox's public key $PU(M)$ to make sure the middlebox is *legitimate*. Then, the server takes several steps to make sure the middlebox is *honest*.

- **Step 1.** Calculate the hash value of the first h packets' contents and compare it with the field *Hash value of first h packets* in the *ProofCert*. These two values must be identical.
- **Step 2.** Classify f with the corresponding model indicated by the model label. The classification result must be positive.

Through steps 1 and 2, one can prevent the middlebox M from labeling flows arbitrarily. If anyone of these steps fails, the verification server returns an error to M and aborts the whole process. Otherwise, the verification server classifies f with other models uploaded by different middleboxes or deployed by itself to test the *reliability* of M . The classification results and the descriptions of the corresponding models are appended to the original *ProofCert* (*Result_i* and *Model_name_i* in Fig. 2). Finally, the verification server signs the extended *ProofCert* with its private key $PR(V)$ and returns it as the final evidence to the middlebox M .

4.3. Evidence verification

Once the middlebox receives the final evidence (i.e., the extended *ProofCert*), it is to provide the *ProofCert* to the endpoint and asks for the corresponding session-key. For this purpose, first, the middlebox should make a network connection to the endpoint. However, this is challengeable because the endpoint may be behind NAT and use a private IPv4 address. This is common in today's Internet. To solve this issue, we design two special notification mechanisms to let the endpoint

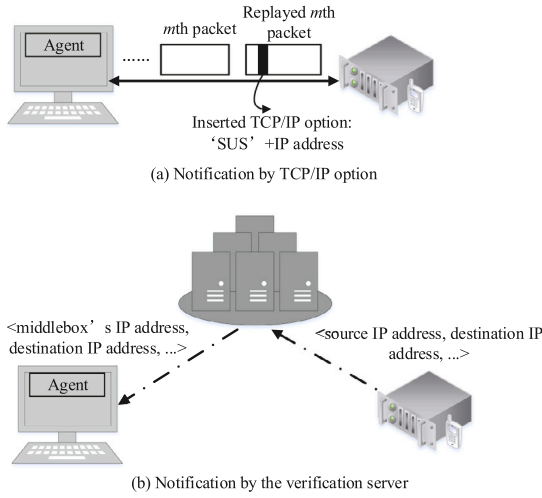


Fig. 3. Two different notification mechanisms.

connect to the middlebox actively, as depicted in Fig. 3. The details are as follows.

Suppose that the middlebox uses the first m packets of flow f for classification and that the classification result is positive. The m th packet is the incoming packet, i.e., the packet returned from the application server. In order to inform the endpoint, the middlebox replicates the m th packet, inserts an indicator “SUS” and its IP address in the TCP/IP header options (the best place is the TCP option), and replays the packet. When the agent observes an incoming packet, it first checks the TCP/IP header options. If an item matches the style “SUS”+IP address, the agent drops this packet and establishes a TLS connection to the IP address, namely, the middlebox.

If the m th packet is the last and flow f is already terminated, a firewall may drop the replicated packet. The router may also drop the packet if it does not support TCP/IP options. In such cases, the middlebox has to inform the endpoint with the help of the verification server. It sends f 's source IP address, destination IP address, destination port and the hash value of the first h packets' contents with an authentication code to the verification server. Note that agents have established long-lived TLS connections with the verification server upon start-up, and those behind the same NAT may have the same public address. The verification server searches connections with the same source IP address, and sends the $\langle$ middlebox's IP address, f 's destination IP address, destination port, hash value, and authentication code $\rangle$ to the corresponding agents. The agent indeed generated flow f (as confirmed by having the same destination IP address, destination port, and hash value) is to establish a TLS connection to the middlebox with the authentication code.

The first notification method is simple and easy to implement, so it is preferred in ME-Box. When it fails, for example, when the waiting time expires, the middlebox turns to the second method. In the second method, the authentication code is actually the agent's dynamic password. Only an authenticated agent can connect to the middlebox and perform the next evidence verification step. In the implementation, the authentication code can be a random string, and it is used as the preshared key (PSK) in the TLS handshake (the key exchange mode can be PSK with DHE).

The agent is always monitoring local network connections, similar to a personal firewall. For each encrypted flow, the agent saves the destination IP address, destination port, and the hash value of the first h packets' contents in memory for a fixed short time, for instance, 30 min. This information is used to label an encrypted flow and verify the received evidence, and when the verification is completed or the set time is up, the corresponding flow information will be deleted. The evidence verification procedure is described as follows.

When the agent receives the notification, it creates a TLS connection to the middlebox. In the TLS handshake process, the middlebox sends its X.509 certificate to the agent as usual. The agent first checks the X.509 certificate of the middlebox. If the certificate is valid and signed by some trusted CA (Certificate Authority), the verification continues. Otherwise, the agent disconnects the connection. Once the TLS connection is established successfully, the middlebox sends the evidence to the agent. The agent verifies the received evidence (i.e., the *ProofCert*) in the following steps. It first searches the saved encrypted flow information according to the destination IP address, destination port and hash value (*Dst IP*, *Dst port* and *Hash value of first h packets* in the *ProofCert*). If some flow is matched, which means that the endpoint indeed generates the flow, the agent then checks the signature value *Signature_V* by the verification server's public key *PU(V)*. If the signature checking is passed, the agent can believe that the evidence is real and integrated. Finally, the agent makes a decision based on *Result_i* ($i = 1, 2, \dots, r$) contained in the *ProofCert*. The decision-making method is elaborated in the following subsection.

4.4. Confirmation by DPI

As described in Section 4.2, the verification server classifies the suspicious flow f with additional r models. Suppose that models $m_p^1, m_p^2, \dots, m_p^p$ make positive classification decisions, and that models $m_n^1, m_n^2, \dots, m_n^n$ produce negative results ($p + n = r$). Model m_p^i has a detection rate (true positive rate) d_p^i and a weight w_p^i , while model m_n^j has a detection rate d_n^j and a weight w_n^j . Therefore, if

$$\sum_{i=1}^p (d_p^i \times w_p^i) > \sum_{j=1}^n (d_n^j \times w_n^j), \quad (3)$$

the agent should choose to trust the middlebox M 's detection result, namely, the encrypted flow is malicious. Particularly, if all models have the same detection rate and the same weight, formula (3) is reduced to

$$p > n. \quad (4)$$

For simplicity, we assume that if $p > n$, the agent accepts the detection result and sends the session-key to the middlebox.

The last thing we should do now is extracting the session-key from the endpoint. In this work, we mainly focus on TLS traffic, and there are several ways to accomplish this. For example, we can configure applications such that they will save the session-keys in local files. This is mainly feasible for the Firefox and Chrome browsers. We can also extract the master secrets from memory as done in [36,44], and the session-key can be derived from the master secrets [45]. When the agent observes a TLS client hello packet [46], it records the parameter *client random* and also the parameter *server random* returned from the application server. Once the TLS handshake is finished, the agent dumps the system memory and searches master secrets according to regular expressions, such as `\x35\x6c\x73\x73(?=(\x02\x00|[\x00-\x03]\x03)\x00\x00.{4}.{8}.{4}).{48})` for Microsoft Schannel [36]. To make this process more efficiently, we can further interpose the library, such as *libssl.so* (for Linux systems) or the process *lsass.exe* (for Windows systems), so that the master secrets can be read directly or that only the memory of *lsass.exe* should be dumped.

The third method is to integrate a TLS MITM proxy in the agent. This is a lightweight approach that is applicable for almost all applications. Since the agent only works locally, the MITM proxy will not threaten end-users' privacy. In our prototype implementation, all of these three methods are supported so that we can extract session-keys from the endpoint comprehensively. With the session-key, the middlebox can decrypt a suspicious encrypted flow and perform DPI with Snort² or Suricata³ to detect attacks. The final detection result

² <https://www.snort.org/>.

³ <https://suricata-ids.org/>.

will be fed back to the verification server to construct labeled encrypted traffic datasets.

5. Analysis of ME-Box

Defending against dishonest middleboxes. The core design of ME-Box is to resist dishonest middleboxes. The dishonest middleboxes can be real in practice; for instance, totalitarian governments deploy TLS middleboxes to monitor social media [47]. In such cases, when a middlebox observes an encrypted flow connected to some particular target such as *Facebook*, it can first classify the flow as malicious deliberately. Then it requests the session-key from the endpoint to decrypt the content and monitor users' social network behaviors. To defend against such dishonest middlebox, ME-Box introduces verification servers to reclassify the suspicious flow with the same model used by the detecting middlebox and the models adopted by other middleboxes. If the reclassification result is negative, namely, if the flow is judged as normal by the majority, the agent will not send the session-key, thus defeating dishonest claims of suspicious encrypted flows.

Note that several dishonest middleboxes can work together to deceive end-users. According to formula (4), if $p > n$, the agent will send the session-key to the requesting middlebox. Therefore, if most of the middleboxes are dishonest and they judge some of the same specific flows as suspicious, the agent will be fooled to believe the detection results. Suppose that honest middleboxes have a same true negative rate t_n , the maximum number of collaborating dishonest middleboxes we can allow is

$$\max(0, \frac{(2t_n - 1) \times (p + n)}{2t_n}). \quad (5)$$

The derivation and analysis of formula (5) is in [Appendix](#).

Defending against passive and active attacks. In ME-Box, we use the TLS protocol to encrypt all connections. Therefore, the confidentiality and integrity of the network traffic are guaranteed. An attacker cannot get the key or flow content information by observing the network packets passively. An attacker can also design several active attacks to steal session-keys. For example, an attacker can first take control of the middlebox through software vulnerabilities. Afterward, he can replace the original classification model with a weaker one and upload the weaker model to the verification server. The weaker model will classify all encrypted flows as suspicious, and the attacker might obtain all the session-keys. However, this is almost impossible because the verification server also tests suspicious flows on other models. The theoretical analysis is the same as the *Defending against dishonest middleboxes*.

The attacker may target the agent and illegally access the session-keys by compromising the endpoint system. As pointed out in the threat model (Section 3.3), one can utilize Intel SGX to guarantee the security of session-keys. In fact, if an attacker has compromised the endpoint system, he does not need to steal these keys. He can observe all the plaintext data by easily hijacking system calls. Therefore, the agent will not add more risks to the security and privacy of end-users. If the verification server is attacked, an attacker can steal the uploaded flows and models. However, they are all encrypted, so the information leakage should be limited. In the verification server, an attacker may also fool the endpoints or middleboxes by generating arbitrary evidence. However, the evidence needs to be signed by the correct private key $PR(V)$, and it is almost impossible for the attacker acquiring the key, as assumed in our threat model. Hence, the evidence cannot be signed correctly, and the type of attacks can be detected efficiently.

Impacting on endpoints. In general, ME-Box introduces a small performance overhead to endpoints. The main tasks of the agent installed on the endpoint are to calculate the hash value of the first h packets and extract the session-key for each encrypted flow. These operations are ordinary and can be implemented efficiently. Thus, the

agent consumes a very small amount of CPU usage ($< 7\%$) when processing encrypted traffic. The saved hash values and session-keys are quite a few bytes and to be deleted after a period of time (e.g., 30 mins). Therefore, the consumed memory is also small. We further describe these performance results in Section 6.3.

6. Evaluation

6.1. Experimental setup and system implementation

Experimental setup. The agent is installed on a Windows 7 virtual machine. The tested malware is also run on this virtual machine. The middlebox is a Thinkpad laptop with 4 GB memory and Pentium Dual-Core CPU T4500. It is running Ubuntu 16.04. We configure the laptop as a hotspot, and the virtual machine connects to the Internet by the hotspot. Therefore, the middlebox can observe all the traffic sent and received by the endpoint. The verification server is a Dell PowerEdge R730 server running CentOS 7. It has 32 processors, 16 GB RAM and a 12 TB hard drive. The malware is downloaded from Virusshare.⁴

Implementation. We implement the user agent program in C++. In the program, we use *wincap* to capture and check network packets. To be specific, when the agent observes a TCP SYN packet, it records the current time, source and destination IP addresses, as well as the source and destination ports as the indicators for this TCP flow. After the three-way handshake, the agent parses the first data packet to check whether it is a TLS Client Hello message, and if so, it calculates the hash value of the first 10 packets' contents. The hash value is saved and indexed by the TCP flow's indicator in memory, and the expiration time is set to 30 min.⁵ If it is not, the following packets are passed directly. For TLS traffic, the incoming packets (from the application server to the endpoint) are additionally checked for the presence of a specific TCP option: "SUS"+IP address.

In addition to traffic capturing, the agent's other main function is to monitor and extract applications' TLS session-keys. We provide several ways to accomplish this. The SSLKEYLOGFILE environment variable is created to record the client randoms and the master keys generated by the Firefox and Chrome browsers. A global proxy (*Proxifier*⁶+*mitmproxy*⁷) is set up to extract these parameters for other applications. If an application (e.g., the IE browser) refuses to accept the global proxy's X.509 certificate, we also provide an option to inject the process *lsass.exe* (it generates client randoms and master keys for Window applications) and dump its memory to search the client randoms and the master keys. The injection code is borrowed from the Cuckoo Sandbox.⁸ By these methods, we can efficiently obtain the TLS session-keys at the endpoint. The agent also contains OpenSSL libraries to calculate hash values of flows and establish TLS connections with the middlebox and the verification server.

The middlebox is implemented by Python. To detect malicious traffic in nearly real time, the middlebox only analyzes at most the first 100 packets for each TCP flow. It uses *tshark*⁹ to capture network traffic and save it as pcap files. These pcap files are processed by *Joy*¹⁰ to extract the classification features for each TLS flow. The classification algorithm is the decision tree provided by Scikit-learn. If the classification result is suspicious, the middlebox uses *Scapy*¹¹ to insert a TCP option ("SUS"+IP address) into the 100th packet (the flow must have more than 100 packets) and replay it to the endpoint.

⁴ <https://virusshare.com/>.

⁵ We set the expiration time to 30 min to make sure that the agent has sufficient time to receive and verify the *ProofCert*.

⁶ <https://www.proxifier.com/>.

⁷ <https://mitmproxy.org/>.

⁸ <https://github.com/cuckoosandbox/community>.

⁹ <https://www.wireshark.org/docs/man-pages/tshark.html>.

¹⁰ <https://github.com/cisco/joy>.

¹¹ <https://scapy.net/>.

Table 2
Classification feature set.

Flow metadata	The number of inbound/outbound bytes and packets The source and destination ports The total duration of the flow in seconds
Packet lengths and times	Sequence of packet lengths and packet inter-arrival times
Byte distribution	Probabilities for each byte value in the packets' payloads, which are computed by dividing the byte distribution counts by the total number of bytes.
TLS handshake information	List of offered ciphersuites List of advertised extensions Public key length Whether the server certificate is self-signed or not

As noted in Section 4.2, this notification mechanism may fail. If the endpoint has no response in 5 min, the middlebox sends the *< source's IP address, destination IP address, destination port, hash value, and authentication code>* to the verification server for forwarding. This is also the notification mechanism for flows with less than 100 packets. In the middlebox, OpenSSL is invoked to calculate hash values, generate random numbers and create TLS connections.

The verification server is implemented as a web server. It provides a web page for middlebox registration. It also provides web pages to index uploaded malicious flows. The flow indexing is realized with the help of Moloch.¹² In the background, it uses Scikit-learn to implement several different classification models, such as decision tree, random forest and support vector machine (SVM). We chose these 3 machine learning algorithms because they have been tested in commercial environments and achieved the best classification results [10]. The trained models' parameters of these algorithms are included in python code that is compiled and encrypted as .pye files. Communications with the agent and the middlebox are conducted by WebSocket, which are encrypted by the TLS protocol. The connection information is saved in a SQLite database for quick searching.

6.2. Functional evaluation

We downloaded more than 5000 windows malware files from Virusshare by searching with the keyword "windows". To train the classification models, we install and run these malware files on the Windows 7 virtual machine one by one. Each malware is run for 30 min, and the traffic is captured by Wireshark. When the time is up, we restore the virtual machine and run the next malware. In total, we get 1754 TLS flows as malicious samples, which are generated by 538 malware programs. The normal traffic is generated by web browsing. We visit the Alexa Top 500 sites¹³ using Firefox on the virtual machine, and we use Wireshark to capture the network traffic. From the captured traffic, We choose the first 1754 TLS flows as the normal samples. The malicious and normal samples are processed by Joy to extract the classification features, which are listed in Table 2. These classification features are proposed in literature [11], and we directly use them in here because the main concern of this work is to design a new framework for malware detection other than a specific detection method. The features preprocessing is the same as in [11]. In the process of generating network traffic, all client randoms and master keys are saved in the NSS key log format.

For testing the proposed method, we use 1600 malicious flows and the same number of normal flows as the training dataset. The remaining 308 flows (154 malicious flows + 154 normal flows) are used as the testing dataset. The middlebox uses the decision tree (an

Table 3
Detection results without decryption.

Detected	Actual	
	Malware's flows	Alexa top websites' flows
Suspicious	148	15
Normal	6	139

Table 4
Confirmation results by decrypting and signature matching.

Confirmed by Suricata	Detected by machine learning	
	148 malware's flows	15 false positives
Malicious	148	0
Normal	0	15

optimized version of the CART algorithm provided by Scikit-learn) as the classification algorithm. In the verification server, the deployed classification models are decision tree, random forest and SVM. The detection results without decryption (i.e., detected by the middlebox through the decision tree algorithm and verified by the verification server) are shown in Table 3. As shown in the table, the true positive rate is 96.1%, and 15 flows are false positives. We analyze these false positives manually by downloading the corresponding webpages. The webpages' HTML codes are read carefully, and we confirm that they are safe.

We decrypt the detected 163 suspicious flows (148 malicious flows + 15 false positives) using the corresponding client randoms and master keys. The decrypted flows are further confirmed by Suricata with open emerging threats rules. The confirmation results are listed in Table 4. Note that the 15 false positives are all excluded. This result implies that our method can reduce the false positives of machine learning algorithms and produce more accurate samples of maliciously encrypted flows. Our method needs to decrypt 163 flows, and among them, only 15 flows may threaten end-users' privacy. Compared to the decryption followed by DPI methods, the number of decrypted flows is significantly reduced (the former methods need to decrypt all 308 testing flows), and the end-users' privacy can be better protected. Compared to machine learning methods, our method is more accurate because all false positives are excluded.

To simulate a dishonest middlebox, we add an extra rule on the decision tree model: if a connection is established to Facebook (i.e., the domain name is **facebook.com*), it is judged as suspicious. We visit Facebook several times and test whether ME-Box can prevent such malicious models. The experimental results show that the random forest and SVM models always classify the traffic as normal, so there are two negative classification results in the returned *ProofCert*. According to formula (3), the agent does not need to give the session-keys ($p = 1$ and $n = 2$ in this case), thus the end-user's privacy is protected.

6.3. Performance evaluation

Since the middlebox and the verification server can be run on dedicated servers, we mainly evaluate the performances of the agent and the time spreading for malware detection in our experiments. As described in Sections 4.3 and 6.1, the core functions of the agent are packet inspection (including find out TLS traffic and calculate TLS flows' hash values), key extraction and evidence verification. Generally, the more (malicious) encrypted traffic, the more burden an agent will bear. We test the agent with various numbers of encrypted network flows and compare the CPU, memory, communication overhead and detection time performances with those of the latest Direct DPI on ciphertext method PE-DPI [8]. Similar to our method, PE-DPI also use two non-collusion servers for malicious encrypted traffic detection.

In PE-DPI, the sender needs to transform the payloads of packets into a series of tokens and encrypt the tokens with the Boneh-Goh-Nissim cryptosystem. This consumes a lot of CPU resources. As shown

¹² <https://molo.ch/>.

¹³ <https://www.alexa.com/topsites>.

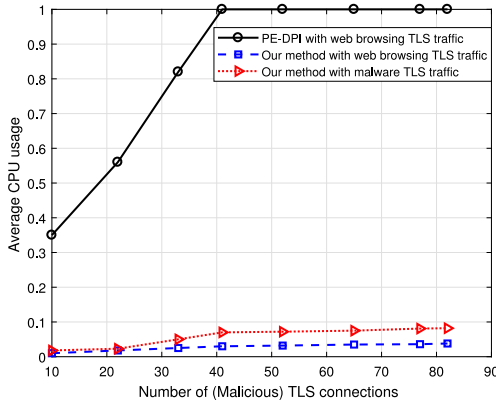


Fig. 4. Comparison of CPU consumption. We tested several (malicious) TLS flows and recorded the corresponding CPU consumptions by *typeperf* tool. The whole process has been repeated 40 times to finally calculate the average CPU consumptions. We also have calculated its confidence interval for each average value in the figure at the 95% confidence level. For PE-DPI, the confidence interval of CPU consumption is [0.33, 0.37] when dealing with 10 TLS flows. The confidence interval is [0.036, 0.040] for our method with 82 web browsing TLS traffic, and the confidence interval is [0.080, 0.084] for our method with 82 malicious TLS traffic.

in Fig. 4, when dealing with 10 TLS connections at the same time (accessing only 4 websites using Firefox), PE-DPI consumes average 35% of the CPU (the confidence interval is [0.33, 0.37] at the 95% confidence level). When the endpoint sends 41 TLS connections simultaneously, the CPU utilization is 100%, while the CPU usages of our method are significantly lower. When processing the web browsing TLS traffic, the highest average CPU usage is only 3.8% (the confidence interval is [0.036, 0.040] at the 95% confidence interval). This is because the TLS keys are saved by the Firefox browser automatically, and the agent does not need to extract them. Additionally, it does not need to establish TLS connections with the middlebox for evidence verification. Therefore, the CPU overhead is low. Upon encountering malware's TLS traffic, the agent still consumes a small number of CPU resources. The highest average CPU usage is only 8.2% (the confidence interval is [0.080, 0.084] at the 95% confidence interval). Fig. 5 depicts the average memory consumptions of PE-DPI and our method. Both methods require a small amount of memory (approximately 40M Bytes when dealing with 82 flows) resources because there are a few data needs to be stored in the memory.

The agent's communication overheads are negligible. If there is no malware traffic, the communication overhead is zero. Namely, the agent will not set up any TLS connection. If there is one malicious flow, the agent needs to connect to the middlebox for receiving the *ProofCert* and possibly, sending the session-key. Therefore, if there are i malicious flows, the agent's communication overheads are: $i \times (\text{one TLS connection} + \text{a ProofCert} + \text{session-keys})$. We use message bytes to measure communication overheads. The overheads of the TLS connection are the bytes of handshake messages. For *ProofCert*, we use 4 bytes for the field *IdentM*, 4 bytes for *Model label*, 4 bytes for *Dst IP*, 2 bytes for *Dst Port*, 64 bytes for the hash value (SHA512), 128 bytes for *Signature_M* (RSA 1024), 4 bytes for the *Model_name_i* field, 1 byte for *Result_i*, and 64 bytes for the last field *Signature_V*. The client random and the master key are 80 bytes (32+48). The communication overheads with various numbers of malicious flows are shown in Fig. 6. Since PE-DPI splits packets and sends the encrypted tokens to the middlebox for inspection, its communication overheads are at least the size of the traffic contents. The communication overheads of PE-DPI are also shown in Fig. 6, and obviously, the proposed method imposes very small overheads to the endpoint's network. For example, if the endpoint is infected by a malware program, and the malware generates 3 TLS connections, the total communication overhead is only 9.4 KB, which is a small amount and can be ignored.

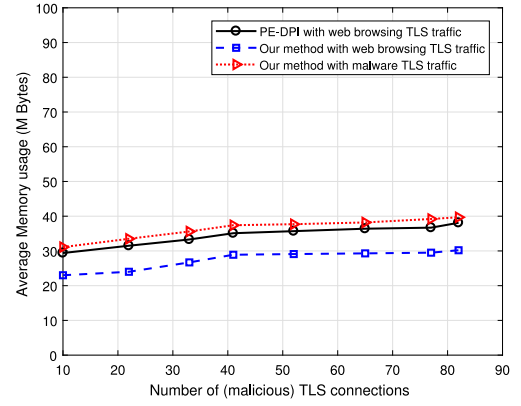


Fig. 5. Comparison of memory consumption. Similar to Fig. 4, We tested several (malicious) TLS flows and recorded the corresponding Memory consumptions by *typeperf* tool. We repeated the experiments 40 times to calculate the average values and also the confidence intervals at the 95% confidence level. When the number of TLS flows is 82, the confidence interval is [35.7, 40.5] for PE-DPI, and is [28.3, 32.1] for our method with web browsing TLS traffic. Our method with malware TLS traffic has the highest memory consumptions, and the confidence interval is [34.8, 44.6] when the flow number is 82.

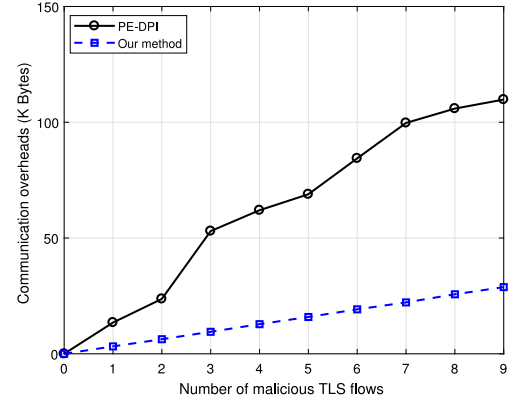


Fig. 6. Comparison of communication overheads. For PE-DPI, we repeated the experiments 40 times to calculate the average values and the confidence intervals at the 95% confidence level. When the number of flows is 9, the communication overhead for PE-DPI is 109.8 KB, and the confidence interval is [102.9, 116.7]. Note that the communication overheads of our method are deterministic. If there is i flows, the communication overheads is $i \times (\text{one TLS connection} + \text{a ProofCert} + \text{session-keys})$ bytes. In our experiments, the value of (one TLS connection + a *ProofCert* + session-keys) is 3.2 KB.

Table 5

Average detection time.

	PE-DPI	Our method
Average detection time	3 h	16.2 s

The average detection times compared to those of PE-DPI are listed in Table 5. We test 10 malware files one by one on the Windows 7 virtual machine. For our method, the detection time is calculated as the time confirmed by the Suricata minus the flow starting time. For PE-DPI, the detection time is the time taken for the first matching of the encrypted keywords and the encrypted tokens. There is a total of 25K Suricata rules, and the matching takes an average of 3 h, while the average detection time of our method is only 16.2 s. Thus, ME-Box can detect maliciously encrypted flows in a much quicker manner, and it is suitable to be deployed in real networks.

7. Discussion

In this section, we discuss several possible deployments and the limitations of ME-Box. One can implement and deploy ME-Box as a standalone system. Besides, our proposed method can be integrated with existing solutions. In practice, a possible example is the Cisco Encrypted Traffic Analytics (ETA) [48]. Cisco ETA uses proxy servers, endpoint telemetry, NetFlow, traffic segmentation and much more to establish normal behavior of hosts and users in an enterprise. Based on these, it further uses multi-layer machine learning methods to detect malicious encrypted traffic. One can upgrade the endpoint telemetry to extract applications' session-keys and examine the detection results from the Stealthwatch (a core component of Cisco ETA that performs machine learning algorithms). The Stealthwatch further decrypts the detected malicious encrypted flows for confirmation. Consequently, the final detection results can be more accurate and more convincing. This may promote the acceptance of existing solutions in various application areas, such as the Industrial Internet of Things.

In ME-Box, a third-party server, i.e., the verification server, is introduced to validate the results of machine learning algorithms. This may limit the practical use of ME-Box. However, we argue that the added verification server has a potential advantage for network security researches and applications. That is, it can be a unified platform for network security data collection and machine learning model testing. The verification server can provide labeled malicious traffic to train different detection models, and different models can be deployed in the server to be fairly compared. The large labeled dataset and fair comparison can make the machine learning algorithms more practical and acceptable for use in network security protection.

Furthermore, one can use the Blockchain technology [49] to replace the functions of verification servers. With the blockchain, all middleboxes can be designed to be miners and they will validate the classification results for each other. For instance, the classification results and the suspicious flows are saved in the blocks, and all other middleboxes can test it with its own classification model. The middleboxes who successfully verify them will be rewarded with some digital currency or tokens. The verification results are also written to the blocks, and endpoints can read these verifications as historical data to estimate the reputation of a middlebox. We defer this to our future work.

8. Conclusion

In this paper, we propose ME-Box, a novel framework and system used to detect malicious encrypted traffic. ME-Box is composed of agents, middleboxes and verification servers. The middlebox uses machine learning methods to estimate whether or not an encrypted flow is suspicious. Because the middlebox may be dishonest and the estimation may be intentionally manipulated, to solve this issue, a particular *ProofCert* and a complete verification process is designed. We introduce the verification server to validate the *ProofCert* by classifying the malicious flows using several different classification models. Notably, the verification server can also be a unified platform for malicious traffic collection and model testing. This may promote the adoption of machine learning methods and produce new opportunities for network security research and applications.

We have implemented a prototype system of ME-Box and tested it using real malware. The experimental results show that ME-Box requires no modification of current cryptographic protocols. It also maintains end-users' privacy well, and its performance is practically deployable. In the future, we will design and implement a non-centralized evidence verification mechanism discussed in Section 7.

CRedit authorship contribution statement

Bingfeng Xu: Conceptualization, Methodology, Writing - original draft, Investigation, Software, Validation. **Gaofeng He:** Conceptualization, Methodology, Software, Writing - review & editing. **Haiting Zhu:** Resources, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported by National Natural Science Foundation of China under grants 61802192 and 61702282, by Natural Science Foundation of the Jiangsu Higher Education Institutions of China under the grant 18KJB520024, by Nanjing Forestry University (GXL016, CX2016026), by NUPTSF under the grant NY217143.

Appendix. Derivation and analysis of formula (5)

Denote the number of dishonest middleboxes as m , and the total number of middleboxes as N . m dishonest middleboxes work together to deceive end-users, i.e., their models can generate same results for some specific flows. Flow f is a benign encrypted flow, and the dishonest middlebox wants to obtain the session-key of f . Thus, m dishonest middleboxes classify f as suspicious. Since the true negative rates of honest middleboxes are t_n , they classify f as suspicious with probability $1 - t_n$. Therefore, the expected value of p (positive results returned from the verification server) is

$$p = m + (N - m) \times (1 - t_n). \quad (A.1)$$

The expected value of n (negative results returned from the verification server) is

$$n = (N - m) \times t_n. \quad (A.2)$$

According to formula (4), if $p > n$, f is accepted as malicious by the end-point, and the agent will send f 's session-key to the dishonest middlebox. Therefore, we have

$$\begin{aligned} m + (N - m) \times (1 - t_n) &> (N - m) \times t_n \\ \Rightarrow m &> \frac{(2t_n - 1) \times N}{2t_n} = \frac{(2t_n - 1) \times (p + n)}{2t_n}. \end{aligned} \quad (A.3)$$

Since $m \geq 0$, in order for ME-Box to work properly, the maximum number of dishonest middleboxes allowed is

$$AM = \max(0, \frac{(2t_n - 1) \times (p + n)}{2t_n}). \quad (A.4)$$

From formula (A.4), one can observe that if $t_n = 1$, $AM = \frac{(p+n)}{2}$, namely, an attacker has to control more than half of middleboxes to compromise ME-Box. If $t_n \leq 0.5$, $AM = 0$. This implies that every encrypted flow will be judged as suspicious, even though all the middleboxes are honest. Therefore, the adopted machine learning methods should have high true negative rates, or equivalently, low false positive rates.

References

- [1] Maddison J. More encrypted traffic than ever. 2018, <https://www.fortinet.com/blog/industry-trends/more-encrypted-traffic-than-ever.html>.
- [2] Symantec Security Response. IoT devices being increasingly used for DDoS attacks. 2016, <https://www.symantec.com/connect/blogs/iot-devices-being-increasingly-used-ddos-attacks>.
- [3] Cisco. Detect encrypted malware traffic and secure network. 2019, https://www.cisco.com/c/en_us/solutions/enterprise-networks/network-refresh-guidance/secure-your-network-by-detecting-encrypted-malware-traffic-with-machine-learning.html.
- [4] Naylor D, Schomp K, Varvello M, Leontiadis I, Blackburn J, López DR, Pagiannaki K, Rodriguez Rodriguez P, Steenkiste P. Multi-context TLS (mcTLS): Enabling secure in-network functionality in TLS. ACM SIGCOMM Comput Commun Rev 2015;45(4):199–212.

- [5] Liu C, Cui Y, Tan K, Fan Q, Ren K, Wu J. Building generic scalable middlebox services over encrypted protocols. In: IEEE INFOCOM 2018-IEEE conference on computer communications. IEEE; 2018, p. 2195–203.
- [6] Sherry J, Lan C, Popa RA, Ratnasamy S. Blindbox: Deep packet inspection over encrypted traffic. ACM SIGCOMM Comput Commun Rev 2015;45(4):213–26.
- [7] Canard S, Diop A, Kheir N, Paindavoine M, Sabt M. Blindids: Market-compliant and privacy-friendly intrusion detection system over encrypted traffic. In: Proceedings of the 2017 ACM on asia conference on computer and communications security. ACM; 2017, p. 561–74.
- [8] Li H, Ren H, Liu D, Shen XS. Privacy-enhanced deep packet inspection at outsourced middlebox. In: 2018 10th International conference on wireless communications and signal processing. IEEE; 2018, p. 1–6.
- [9] Anderson B, McGrew D. Identifying encrypted malware traffic with contextual flow data. In: Proceedings of the 2016 ACM workshop on artificial intelligence and security. ACM; 2016, p. 35–46.
- [10] Anderson B, McGrew D. Machine learning for encrypted malware traffic classification: accounting for noisy labels and non-stationarity. In: Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining. ACM; 2017, p. 1723–32.
- [11] Anderson B, Paul S, McGrew D. Deciphering malware's use of TLS (without decryption). J Comput Virol Hacking Tech 2018;14(3):195–211.
- [12] Han J, Kim S, Ha J, Han D. Sgx-box: Enabling visibility on encrypted traffic using a secure middlebox module. In: Proceedings of the first asia-pacific workshop on networking. ACM; 2017, p. 99–105.
- [13] Chen Y-C, Li Y-J, Tseng A, Lin T. Deep learning for malicious flow detection. In: 2017 IEEE 28th annual international symposium on personal, indoor, and mobile radio communications. IEEE; 2017, p. 1–7.
- [14] Sommer R, Paxson V. Outside the closed world: On using machine learning for network intrusion detection. In: 2010 IEEE symposium on security and privacy. IEEE; 2010, p. 305–16.
- [15] Gu T, Dolan-Gavitt B, Garg S. Badnets: Identifying vulnerabilities in the machine learning model supply chain. 2017, arXiv preprint arXiv:1708.06733.
- [16] Chen Z, Yan Q, Han H, Wang S, Peng L, Wang L, et al. Machine learning based mobile malware detection using highly imbalanced network traffic. Inform Sci 2018;433:346–64.
- [17] Aublin P-L, Kelbert F, O'Keeffe D, Muthukumar D, Priebe C, Lind J, et al. Tech. rep., Imperial College London; 2017.
- [18] de Carnavalet XdC, Mannan M. Killed by proxy: Analyzing client-end TLS interception software. In: Network and distributed system security symposium. 2016, p. 1–17.
- [19] O'Neill M, Ruoti S, Seamons K, Zappala D. TLS proxies: Friend or foe? In: Proceedings of the 2016 internet measurement conference. ACM; 2016, p. 551–7.
- [20] Lindell Y, Pinkas B. A proof of security of Yao's protocol for two-party computation. J Cryptol 2009;22(2):161–88.
- [21] Yuan X, Wang X, Lin J, Wang C. Privacy-preserving deep packet inspection in outsourced middleboxes. In: IEEE INFOCOM 2016-the 35th annual IEEE international conference on computer communications. IEEE; 2016, p. 1–9.
- [22] Fuhr T, Paillier P. Decryptable searchable encryption. In: International conference on provable security. Springer; 2007, p. 228–36.
- [23] Boneh D, Goh E-J, Nissim K. Evaluating 2-DNF formulas on ciphertexts. In: Theory of cryptography conference. Springer; 2005, p. 325–41.
- [24] Desmoulin N, Fouque P-A, Onete C, Sanders O. Pattern matching on encrypted streams. In: International conference on the theory and application of cryptology and information security. Springer; 2018, p. 121–48.
- [25] Kovanen T, David G, Hämäläinen T. Survey: Intrusion detection systems in encrypted traffic. In: Internet of things, smart spaces, and next generation networks and systems. Springer; 2016, p. 281–93.
- [26] Nisioti A, Mylonas A, Yoo PD, Katos V. From intrusion detection to attacker attribution: A comprehensive survey of unsupervised methods. IEEE Commun Surv Tutor 2018;20(4):3369–88.
- [27] Hellemons L, Hendriks L, Hofstede R, Sperotto A, Sadre R, Pras A. SSHCure: a flow-based SSH intrusion detection system. In: IFIP International conference on autonomous infrastructure, management and security. Springer; 2012, p. 86–97.
- [28] Barati M, Abdullah A, Mahmood R, Mustapha N, Udzir NI. Feature selection for IDS in encrypted traffic using genetic algorithm. In: Proceedings of the 4th international conference on computing and informatics. 2013, p. 279–85.
- [29] He G, Zhang T, Ma Y, Xu B. A novel method to detect encrypted data exfiltration. In: 2014 Second international conference on advanced cloud and big data. IEEE; 2014, p. 240–6.
- [30] Koch R, Golling M, Rodosek GD. Behavior-based intrusion detection in encrypted environments. IEEE Commun Mag 2014;52(7):124–31.
- [31] Aceto G, Ciunzio D, Montieri A, Pescapè A. Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges. IEEE Trans Netw Serv Manag 2019;16(2):445–58.
- [32] Aceto G, Ciunzio D, Montieri A, Pescapè A. MIMETIC: Mobile encrypted traffic classification using multimodal deep learning. Comput Netw 2019;165:106944.
- [33] Aceto G, Ciunzio D, Montieri A, Pescapè A. Toward effective mobile encrypted traffic classification through deep learning. Neurocomputing 2020;409:306–15.
- [34] Shekhawat AS, Di Troia F, Stamp M. Feature analysis of encrypted malicious traffic. Expert Syst Appl 2019;125:130–41.
- [35] Radivilova T, Kirichenko L, Ageyev D, Tawalbeh M, Bulakh V. Decrypting SSL/TLS traffic for hidden threats detection. In: 2018 IEEE 9th international conference on dependable systems, services and technologies. IEEE; 2018, p. 143–6.
- [36] Anderson B, Chi A, Dunlop S, McGrew D. Limitless HTTP in an HTTPS world: Inferring the semantics of the HTTPS protocol without decryption. In: Proceedings of the ninth ACM conference on data and application security and privacy. ACM; 2019, p. 267–78.
- [37] Buczak AL, Guven E. A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Commun Surv Tutor 2015;18(2):1153–76.
- [38] Yueai Z, Junjie C. Application of unbalanced data approach to network intrusion detection. In: 2009 First international workshop on database technology and applications. IEEE; 2009, p. 140–3.
- [39] Costan V, Devadas S. Intel SGX explained.. IACR Cryptol ePrint Arch 2016;2016(086):1–118.
- [40] Garg S, Gentry C, Halevi S, Raykova M, Sahai A, Waters B. Candidate indistinguishability obfuscation and functional encryption for all circuits. SIAM J Comput 2016;45(3):882–929.
- [41] Li L, Zou B, Hu Q, Wu X, Yu D. Dynamic classifier ensemble using classification confidence. Neurocomputing 2013;99:581–91.
- [42] Ryffel T, Trask A, Dahl M, Wagner B, Mancuso J, Rueckert D, Passerat-Palmbach J. A generic framework for privacy preserving deep learning. 2018, arXiv preprint arXiv:1811.04017.
- [43] Xu M, Liu J, Liu Y, Lin FX, Liu Y, Liu X. A first look at deep learning apps on smartphones. In: The world wide web conference. ACM; 2019, p. 2125–36.
- [44] Taubmann B, Frädrieh C, Dusold D, Reiser HP. Tlsx: Harnessing virtual machine introspection for decrypting TLS communication. Digital Invest 2016;16:S114–23.
- [45] Stallings W. Cryptography and network security: principles and practice. Pearson Upper Saddle River; 2017.
- [46] Dierks T, Rescorla E. RFC 5246-the transport layer security (TLS) protocol version 1.2. Internet Eng Task Force 2008.
- [47] aminkhoshnood. Iranian government is censoring sites with universal SSL (cloudflare inc ECC CA-2). 2019, <https://community.cloudflare.com/t/iranian-government-is-censoring-sites-with-universal-ssl-cloudflare-inc-ecc-ca-2/110989>.
- [48] Cisco public. Encrypted traffic analytics. 2018, <https://www.cisco.com/c/dam/en/us/solutions/collateral/enterprise-networks/enterprise-network-security/nb-09-encrytd-traf-anlytcs-wp-cte-en.pdf>.
- [49] Underwood S. Blockchain beyond bitcoin. Commun ACM 2016;59(11):15–7.